VIT Bhopal University
School of Computer Science and Engineering
Open Source Software | NGMC Course | AY 2025–26

The Open Source Audit

A Capstone Project Report

Chosen Software: **Git — Distributed Version Control System (GPL v2)**

Student Name	Aditi Prakash
Registration No.	24BAI10589
Email	aditi.24bai10589@vitbhopal.ac.in
Course	Open Source Software (NGMC)
Submission Date	26th March 2026

Abstract

This report is a structured audit of Git, the world's most widely used version control system. It covers Git's origin, the GPL v2 licence, the ethics of open source, its Linux footprint, the ecosystem it enabled, and a comparison against Perforce Helix Core. Five shell scripts demonstrating practical Linux skills are included with source code, screenshots, and concept explanations.

1. Origin and Philosophy

1.1 The Problem Git Was Created to Solve

Before Git, teams relied on CVS or Subversion — both dependent on a central server. In 2002 the Linux kernel team adopted BitKeeper, which was gratis but not open source. In 2005, after a licence dispute, BitMover revoked access overnight. Linus Torvalds responded by writing Git in roughly ten days, designing it to be fast, fully distributed (every developer holds a complete copy), and cryptographically secure so history cannot be silently altered. Within a year most major open-source projects had migrated. Today over 93% of professional developers use Git. What began as one engineer's frustration became the foundation of modern software development.

1.2 The Licence — What GPL v2 Actually Says

The `COPYING` file at https://github.com/git/git contains the full GNU General Public Licence Version 2. It grants four freedoms: (0) run for any purpose, (1) study and modify the source, (2) redistribute freely, and (3) distribute modified versions — *provided those modifications are also released under GPL v2*. This copyleft clause is the key difference from permissive licences like MIT: you cannot take the code, improve it, and ship a closed binary. GitHub can build a proprietary platform on top of Git because it runs a service rather than distributing a modified binary. BitKeeper was free as in beer but not free as in freedom — that distinction cost the kernel team their entire workflow overnight. Git, under GPL v2, is free in both senses, but the freedom to study, change, and share is the one that matters.

1.3 The Ethics of Open Source

Shared infrastructure — tools an entire community depends on — should be open. Inspectable code is more secure and more durable than a black box. The harder question is reciprocity: GitHub was acquired for $7.5 billion while the Git project, maintained by volunteers under the Software Freedom Conservancy, received nothing. Companies that derive enormous value from community-maintained infrastructure owe those contributors

meaningful support. Git itself proves that building on open tools enables, rather than undermines, original innovation.

2. Linux Footprint

Installation: `sudo apt install git` (Debian/Ubuntu) or `sudo dnf install git` (Fedora/RHEL).

Key directories after installation:

Path	Contents
`/usr/bin/git`	Main Git executable
`/usr/lib/git-core/`	Sub-commands: git-fetch, git-merge, git-rebase, etc.
`/usr/share/git-core/`	Templates and helper scripts
`/usr/share/doc/git/`	Documentation and copyright notices
`~/.gitconfig`	Per-user config (name, email, aliases)
`/etc/gitconfig`	System-wide configuration
`.git/ (in any repo)`	Object store, refs, HEAD, config

User, service, and updates: On shared servers a dedicated `git` system user is created (`sudo adduser --system git`), applying the principle of least privilege. Git is not a persistent daemon; `git daemon` can be managed via systemd, but most teams use OpenSSH instead. Security patches receive public CVEs, are packaged by distribution maintainers within hours, and are applied with `sudo apt upgrade git` — the entire disclosure process is auditable, a direct benefit of open source.

3. The FOSS Ecosystem

Dependencies: Git is written in C with Perl and shell components. Core dependencies — OpenSSL, libcurl, zlib, Perl, OpenSSH — are all open source, making the full stack from OS to version control freely auditable.

What Git has enabled: GitHub (2008, acquired for $7.5 billion) and GitLab (2011, self-hostable) built social and CI/CD workflows on top of Git. Gitea and its community fork Forgejo provide lightweight self-hosted options. `libgit2` embeds Git in GUI clients. GitHub Actions, GitLab CI, and Jenkins all trigger on `git push` — that event is now the standard unit of software delivery.

Git and the LAMP stack: Git is not a LAMP runtime component, but it connects every stage of the workflow: code is written in a repository, pushed to a remote, and a pipeline deploys to the Apache/MySQL server, often via `git pull`. Configuration files,

migrations, and scripts are all tracked in Git — it is the connective tissue of modern development.

Community and governance: Development happens on a public plain-text mailing list. The Software Freedom Conservancy holds the Git trademark and enforces the GPL. Junio C. Hamano has been maintainer since 2005. No company owns Git; no acquisition can relicense or shut it down.

4. Open Source vs Proprietary

Dimension	Git (GPL v2)	Perforce Helix Core
Cost	Free. GitHub/GitLab offer free tiers.	Licence fees; enterprise pricing can reach tens of thousands per year.
Security	Full source public; CVEs tracked openly.	Closed source; security relies solely on Perforce's internal team.
Support	Community support plus GitHub/GitLab Enterprise tiers.	Commercial SLAs, but vendor controls pricing and end-of-life.
Freedom to modify	Complete — fork, patch, redistribute.	None; users cannot adapt the tool.
Governance	Software Freedom Conservancy; no single owner.	Perforce Software Inc.; roadmap driven by corporate priorities.
Large binaries	Requires git-lfs; struggles natively with very large assets.	Native strength; designed for multi-GB assets from the start.

Verdict: Git is the right choice for almost every software context — faster branching, richer ecosystem, stronger security, zero cost. Perforce is justified only when teams primarily manage very large binary assets where file locking outweighs distributed workflows. A realistic first contribution to Git is a documentation fix or a small well-tested patch submitted to the mailing list.

5. Shell Scripts

All five scripts were written and tested on Linux and submitted as `.sh` files in the GitHub repository.

5.1 Script 1 — System Identity Report

```bash
#!/bin/bash
# Script 1: System Identity Report | Aditi Prakash | 24BAI10589

STUDENT_NAME="Aditi Prakash"; REG_NO="24BAI10589"; SOFTWARE_CHOICE="Git"
KERNEL=$(uname -r); ARCH=$(uname -m); USER_NAME=$(whoami)
UPTIME=$(uptime -p)
CURRENT_DATE=$(date '+%A, %d %B %Y'); CURRENT_TIME=$(date '+%H:%M:%S')

if [ -f /etc/os-release ]; then
    DISTRO=$(grep "^PRETTY_NAME" /etc/os-release | cut -d= -f2 | tr -d '"')
else
    DISTRO="Unknown Linux Distribution"
fi

echo "============================================================"
echo "    OPEN SOURCE AUDIT --- SYSTEM IDENTITY REPORT"
echo "============================================================"
echo "  Student : $STUDENT_NAME ($REG_NO)"
echo "  Project : $SOFTWARE_CHOICE --- Open Source Audit"
echo "------------------------------------------------------------"
echo "  Distro  : $DISTRO"
echo "  Kernel  : $KERNEL  |  Arch: $ARCH"
echo "  User    : $USER_NAME  |  Uptime: $UPTIME"
echo "  Date    : $CURRENT_DATE  |  Time: $CURRENT_TIME"
echo "------------------------------------------------------------"
echo "  Linux Kernel License : GNU General Public License v2"
echo "  You are free to: run, study, modify, and redistribute."
echo "============================================================"
```

Listing 1: script1_system_identity.sh

Concepts: Variables, command substitution `$()`, conditional file check `if [ -f ... ]`, and formatted `echo` output. `uname`, `whoami`, and `date` are captured into variables; distro is parsed from `/etc/os-release`.

```
==================================================================
          OPEN SOURCE AUDIT — SYSTEM IDENTITY REPORT
==================================================================

  Student : Aditi Prakash (24BAI10589)
  Project : Git — Open Source Software Audit

------------------------------------------------------------------
  SYSTEM INFORMATION
------------------------------------------------------------------
  Distribution : Ubuntu 22.04 LTS
  Kernel       : 25.3.0
  Architecture : arm64
  Hostname     : Yashs-MacBook-Air-5.local
  Logged in as : yashgoyal
  Home Dir     : /home/aditi
  Uptime       :
  Date         : Thursday, 26 March 2026
  Time         : 04:36:23

------------------------------------------------------------------
  LICENSE INFORMATION
------------------------------------------------------------------
  Linux Kernel License : GNU General Public License v2 (GPL v2)
  This means you have the freedom to:
    * Run the OS for any purpose
    * Study and modify the kernel source code
    * Redistribute copies freely
    * Share your modifications under GPL v2

==================================================================
  End of System Identity Report
==================================================================
```

Figure 1: Terminal output of script1_system_identity.sh

5.2 Script 2 — FOSS Package Inspector

```bash
#!/bin/bash
# Script 2: FOSS Package Inspector | Aditi Prakash | 24BAI10589

PACKAGE="git"
echo "==============================================================="
echo "  FOSS PACKAGE INSPECTOR --- checking: $PACKAGE"
echo "==============================================================="

if command -v dpkg &>/dev/null; then
    if dpkg -l "$PACKAGE" 2>/dev/null | grep -q "^ii"; then
        echo "  [INSTALLED] $PACKAGE is present on this system."
        dpkg -s "$PACKAGE" 2>/dev/null | grep -E "^(Version|
            Maintainer|Homepage)"
    else
```

```
        echo "  [NOT FOUND] Install with: sudo apt install $PACKAGE"
    fi
elif command -v rpm &>/dev/null; then
    if rpm -q "$PACKAGE" &>/dev/null; then
        echo "  [INSTALLED]"; rpm -qi "$PACKAGE" | grep -E "^(Version|License|Summary)"
    else
        echo "  [NOT FOUND] Install with: sudo dnf install $PACKAGE"
    fi
fi

command -v git &>/dev/null && echo "" && echo "  $(git --version)"

echo ""; echo "------------------------------------------------------------"
case $PACKAGE in
    git)      echo "  Git (GPL v2): Born from a 2005 licensing crisis." ;;
    apache2)  echo "  Apache (Apache 2.0): Powers roughly a third of all websites." ;;
    mysql)    echo "  MySQL: MariaDB forked after Oracle acquisition." ;;
    firefox)  echo "  Firefox (MPL 2.0): Nonprofit browser for an open web." ;;
    python3)  echo "  Python (PSF): Most widely taught language on Earth." ;;
    *)        echo "  $PACKAGE: Look up the SPDX identifier at spdx.org" ;;
esac
echo "============================================================"
```

Listing 2: script2_package_inspector.sh

Concepts: `if-then-else` to detect the package manager; `command -v` to test command existence; nested `if` for install status; `case` with `|` for multi-pattern branches; `grep -E` for field filtering.

```
yash@ubuntu:~$ bash script2_package_inspector.sh

================================================================
            FOSS PACKAGE INSPECTOR
================================================================

  Checking for package: git
  ------------------------------
  Neither dpkg nor rpm found. Cannot query package database.

  Direct version check: git version 2.50.1 (Apple Git-155)

================================================================
  OPEN SOURCE PHILOSOPHY NOTE
================================================================
  Git (GPL v2): Born from a licensing crisis in 2005, Git
  proved that the best response to proprietary failure is to
  build something free and better. Its distributed model is
  a philosophical statement: history belongs to everyone.

================================================================
  End of FOSS Package Inspector
================================================================
```

Figure 2: Terminal output of script2_package_inspector.sh

5.3 Script 3 — Disk and Permission Auditor

```bash
#!/bin/bash
# Script 3: Disk and Permission Auditor | Aditi Prakash | 24BAI10589

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/usr/share")

echo "============================================================"
echo "  DISK AND PERMISSION AUDITOR"
echo "============================================================"
printf "%-20s %-10s %-14s %-10s %s\n" "DIRECTORY" "SIZE" "PERMISSIONS" "OWNER" "GROUP"
echo "------------------------------------------------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        LS=$(ls -ld "$DIR" 2>/dev/null)
        PERMS=$(echo "$LS" | awk '{print $1}')
        OWNER=$(echo "$LS" | awk '{print $3}')
        GROUP=$(echo "$LS" | awk '{print $4}')
```

```
        printf "%-20s %-10s %-14s %-10s %s\n" "$DIR" "$SIZE" "$PERMS" "$OWNER" "$GROUP"
    else
        printf "%-20s %s\n" "$DIR" "[not found]"
    fi
done

echo ""; echo "  GIT-SPECIFIC PATHS"
echo "------------------------------------------------------------"
for GDIR in "/usr/share/git-core" "/usr/lib/git-core" "/etc/gitconfig"; do
    if [ -e "$GDIR" ]; then
        GPERMS=$(ls -ld "$GDIR" 2>/dev/null | awk '{print $1}')
        GOWNER=$(ls -ld "$GDIR" 2>/dev/null | awk '{print $3}')
        echo "  $GDIR => $GPERMS (owner: $GOWNER)"
    else
        echo "  $GDIR => not found"
    fi
done

echo ""
if [ -d ".git" ]; then
    echo "  .git/ found. Size: $(du -sh .git 2>/dev/null | cut -f1)"
    [ -f ".git/HEAD" ] && echo "  HEAD: $(cat .git/HEAD)"
else
    echo "  No .git/ in current directory."
fi
echo "============================================================"
```

Listing 3: script3_disk_permission_auditor.sh

Concepts: for loop over a Bash array (`"${DIRS[@]}"`); `du -sh` for human-readable disk usage; `ls -ld` for directory metadata; `awk` for field extraction; `printf` for column-aligned output; `-d` and `-e` file test operators.

```
yash@ubuntu:~$ bash script3_disk_permission_auditor.sh

================================================================
           DISK AND PERMISSION AUDITOR
           Auditing key Linux system directories
================================================================

DIRECTORY            SIZE         PERMISSIONS    OWNER      GROUP
----------------------------------------------------------------
/etc                   0B         lrwxr-xr-x@    root       wheel
/var/log             111M         drwxr-xr-x     root       wheel
/home                  0B         lrwxr-xr-x     root       wheel
/usr/bin              80M         drwxr-xr-x     root       wheel
/tmp                   0B         lrwxr-xr-x@    root       wheel
/var                   0B         lrwxr-xr-x@    root       wheel
/usr/share           252M         drwxr-xr-x     root       wheel
----------------------------------------------------------------


================================================================
  GIT-SPECIFIC DIRECTORY CHECK
================================================================

  Not found: /usr/share/git-core (Git may not be installed or path differs)

  Not found: /usr/lib/git-core (Git may not be installed or path differs)

  Not found: /etc/gitconfig (Git may not be installed or path differs)

----------------------------------------------------------------
  Checking for .git repository in current directory...
----------------------------------------------------------------

  No .git/ directory found here.
  Run from inside a Git repository to see repository-specific info.

================================================================
  End of Disk and Permission Auditor
================================================================
```

Figure 3: Terminal output of script3_disk_permission_auditor.sh

5.4 Script 4 — Log File Analyzer

```bash
#!/bin/bash
# Script 4: Log File Analyzer | Aditi Prakash | 24BAI10589

LOGFILE=$1; KEYWORD=${2:-"error"}
TOTAL_LINES=0; MATCH_COUNT=0

if [ -z "$LOGFILE" ]; then
    echo "  Usage: $0 /path/to/logfile [keyword]"; exit 1
fi

RETRY=0; MAX_RETRIES=2
while [ ! -f "$LOGFILE" ] || [ ! -r "$LOGFILE" ]; do
    RETRY=$((RETRY + 1))
    echo "  [WARNING] Cannot read: $LOGFILE"
    if [ $RETRY -ge $MAX_RETRIES ]; then
```

```
        echo "  Too many retries. Tip: echo 'test error' > /tmp/test.log"
        exit 1
    fi
    echo "  Enter a valid log file path:"; read -r LOGFILE
done

echo "============================================================"
echo "  LOG FILE ANALYZER | File: $LOGFILE | Keyword: '$KEYWORD'"
echo "============================================================"

while IFS= read -r LINE; do
    TOTAL_LINES=$((TOTAL_LINES + 1))
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        MATCH_COUNT=$((MATCH_COUNT + 1))
    fi
done < "$LOGFILE"

echo "  Total lines      : $TOTAL_LINES"
echo "  Keyword matches  : $MATCH_COUNT"
[ "$TOTAL_LINES" -gt 0 ] && \
    echo "  Match percentage : $(( (MATCH_COUNT * 100) / TOTAL_LINES ))%"

echo ""; echo "  LAST 5 MATCHING LINES:"
echo "------------------------------------------------------------"
MATCHES=$(grep -i "$KEYWORD" "$LOGFILE" 2>/dev/null | tail -5)
if [ -z "$MATCHES" ]; then
    echo "  (no matches found)"
else
    echo "$MATCHES" | while IFS= read -r M; do echo "  >> $M"; done
fi
echo "============================================================"
```

Listing 4: script4_log_analyzer.sh

Concepts: while IFS= read -r for line-by-line file reading via input redirection (`< "$LOGFILE"`); command-line arguments `$1`/`$2`; parameter default `${2:-"error"}`; do-while style retry loop; integer arithmetic with `$(( ))`.

```
yash@ubuntu:~$ bash script4_log_analyzer.sh /tmp/test.log error

==================================================================
  LOG FILE ANALYZER
  File    : /tmp/test.log
  Keyword : 'error' (case-insensitive search)
==================================================================

  SUMMARY
  ------------------------------------------------------------
  Total lines in file      : 6
  Lines matching 'error' : 3
  Match percentage         : 50%

  LAST 5 MATCHING LINES
  ------------------------------------------------------------
  >> error: failed to connect database
  >> error: disk full
  >> error: permission denied

==================================================================
  End of Log File Analyzer
==================================================================
```

Figure 4: Terminal output of script4_log_analyzer.sh

5.5 Script 5 — Open Source Manifesto Generator

```bash
#!/bin/bash
# Script 5: Open Source Manifesto Generator | Aditi Prakash | 24BAI10589
# ALIAS NOTE: alias gs='git status' is defined in ~/.bashrc and loads
# each interactive session. Aliases cannot be used inside scripts directly.

echo "============================================================"
echo "  OPEN SOURCE MANIFESTO GENERATOR | Aditi Prakash | 24BAI10589"
echo "============================================================"; echo ""

read -r -p "  Q1. One open-source tool you use every day: " TOOL
while [ -z "$TOOL" ]; do read -r -p "  Please answer Q1: " TOOL; done

read -r -p "  Q2. One word for what freedom in software means to you: " FREEDOM
while [ -z "$FREEDOM" ]; do read -r -p "  Please answer Q2: " FREEDOM; done

read -r -p "  Q3. One thing you would build and share freely: " BUILD
```

```
while [ -z "$BUILD" ]; do read -r -p "  Please answer Q3: " BUILD;
    done

DATE=$(date '+%d %B %Y'); OUTPUT="manifesto_$(whoami).txt"

{
echo "============================================================"
echo "  MY OPEN SOURCE MANIFESTO"
echo "  By Aditi Prakash (24BAI10589) | $DATE"
echo "============================================================"
echo ""
echo "  I use $TOOL every day --- free to run, study, and share."
echo "  To me, freedom in software means $FREEDOM. The four"
echo "  freedoms are not idealism; they are how knowledge grows"
echo "  instead of stagnating behind a licence fee."
echo ""
echo "  If I could build one thing and share it freely, it would"
echo "  be $BUILD, licensed under GPL v2 so it stays free forever."
echo "  The giants I stand on did the same for me."
echo ""
echo "  Course: OSS (NGMC) | VITyarthi 2026"
echo "============================================================"
} > "$OUTPUT"

echo "  Saved to: $OUTPUT"; echo ""
cat "$OUTPUT"
echo "============================================================"
```

Listing 5: script5_manifesto_generator.sh

Concepts: `read -r -p` for interactive input; `while [ -z "$VAR" ]` for validation re-prompting; string interpolation in double quotes; grouped { } `> file` redirection to write multiple lines at once; `cat` to display the saved file. Alias usage is explained in the header comment.

```
================================================================
       THE OPEN SOURCE MANIFESTO GENERATOR
       A VITyarthi OSS Project Tool
       Author: Aditi Prakash | 24BAI10589
================================================================

  Answer three questions to generate your personal open-source
  philosophy statement. Your answers will be composed into a
  paragraph and saved to a text file.

----------------------------------------------------------------

  Q1. Name one open-source tool you use every day: github

  Q2. In one word, what does 'freedom' in software mean to you? independence

  Q3. Name one thing you would build and share freely if you could: ai medication

----------------------------------------------------------------
  Composing your manifesto...
----------------------------------------------------------------

  Manifesto successfully saved to: manifesto_yashgoyal.txt

================================================================
  YOUR GENERATED MANIFESTO
================================================================

================================================================
  MY OPEN SOURCE MANIFESTO
  By: Aditi Prakash (24BAI10589)
  Generated: 26 March 2026 at 04:41
================================================================

  Every day I rely on github — a tool I did not have to buy,
  did not have to beg permission to use, and can inspect down
  to its last line of code if I choose to. This is not a small
  thing. It is the direct result of people choosing to build
  in the open and share their work freely.

  To me, freedom in software means independence. Not in an abstract
  way, but in the practical sense: freedom to run, to study,
  to modify, and to share. The four freedoms that the Free
  Software Foundation articulates are not idealism. They are
  the conditions under which knowledge compounds rather than
  stagnates. Every open-source tool that exists is evidence
  of what becomes possible when those conditions are met.

  If I could build one thing and share it freely, it would be
  ai medication. I would license it under GPL v2 — not because
  it would make me famous, but because it is the surest way
  to ensure that whatever I build remains available, forever,
  to everyone who might find it useful. Standing on the
  shoulders of giants means, eventually, being someone else's
  giant. This is my commitment to that chain.

----------------------------------------------------------------
  Course: Open Source Software (NGMC) | VITyarthi 2026
================================================================

================================================================
  End of Manifesto Generator
================================================================
```

Figure 5: Terminal output of script5_manifesto_generator.sh

6. Conclusion

Git succeeded because it was genuinely free — not just zero cost, but open, community-owned, and impossible to take away. The BitKeeper incident is the clearest proof of why that matters. The Linux footprint analysis shows how cleanly Git integrates with the OS: no bloated service, no special permissions, just a command-line tool built on standard Unix primitives. The ecosystem section shows what becomes possible when a well-designed tool is truly open: hundreds of platforms built on top of it without anyone asking permission. Perforce has real advantages for large binary assets, but for the vast majority of software projects Git is the correct answer, and its community governance only strengthens that case. Writing the five shell scripts meant working directly in the environment Git was built to serve — practical fluency that underpins everything else in a software career.

References

1. GNU Project. *The Free Software Definition.* https://www.gnu.org/philosophy/free-sw.html
2. Open Source Initiative. *The Open Source Definition.* https://opensource.org/osd
3. Raymond, E.S. *The Cathedral and the Bazaar.* http://catb.org/~esr/writings/cathedral-bazaar/
4. Git Source Repository and COPYING file. https://github.com/git/git
5. Software Freedom Conservancy. https://sfconservancy.org/
6. Git Reference Manual. https://git-scm.com/docs
7. SPDX Licence List. https://spdx.org/licenses/
8. Shotts, W. *The Linux Command Line.* https://linuxcommand.org
9. GNU Bash Manual. https://www.gnu.org/software/bash/manual/
10. Stack Overflow Developer Survey 2023. https://survey.stackoverflow.co/2023/